

Arquitectura del Sistema de Inventario y Ventas

El presente documento describe la arquitectura de software propuesta para el Sistema de Inventario y Ventas orientado a PYMEs. La solución implementa una arquitectura de tres capas con enfoque modular, permitiendo escalabilidad, mantenibilidad y separación clara de responsabilidades. Adicionalmente, se incorpora la justificación de las decisiones tecnológicas adoptadas en cada capa del sistema.

1. Visión General de la Arquitectura

La arquitectura del sistema se encuentra dividida en tres capas principales: la capa de presentación, la capa de lógica de negocio y la capa de datos. Cada una posee responsabilidades específicas dentro del funcionamiento del software.

Arquitectura del Sistema de Inventario y Ventas

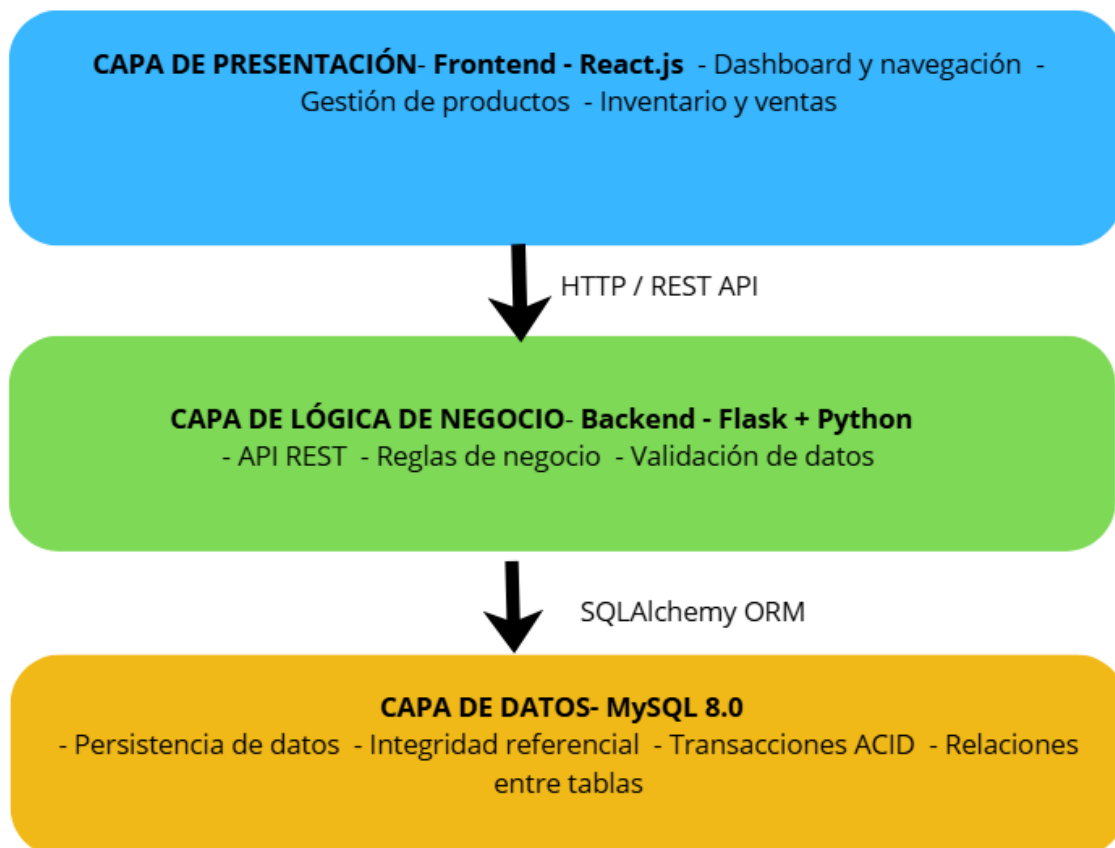


Figura 1. Arquitectura general del sistema.

1.1 Justificación de la Arquitectura

La arquitectura de tres capas fue elegida por ser un patrón ampliamente probado en el desarrollo de sistemas de gestión empresarial. Permite una separación clara de responsabilidades, lo que reduce el acoplamiento entre componentes y facilita el mantenimiento independiente de cada capa. A continuación se detallan las principales razones de esta elección:

- Separación clara de responsabilidades entre frontend, backend y base de datos.
- Facilidad de mantenimiento y actualización de módulos sin afectar otras capas.
- Escalabilidad horizontal para crecimiento de usuarios y volumen de datos.
- Mayor seguridad y control centralizado sobre las operaciones del sistema.
- Compatibilidad con futuras integraciones tecnológicas mediante APIs estandarizadas.

2. Capa de Presentación – Frontend

2.1 Tecnología: React.js

La capa de presentación será desarrollada utilizando React.js, permitiendo interfaces dinámicas, reutilización de componentes y una experiencia de usuario fluida e interactiva.

- Dashboard principal con métricas de ventas y alertas de stock.
- Gestión de productos mediante operaciones CRUD.
- Módulo de inventario y control de stock.
- Registro de ventas y generación de comprobantes.
- Visualización de reportes y gráficos estadísticos.
- Sistema de autenticación y gestión de usuarios.

2.2 Justificación de React.js

React.js fue seleccionado como framework de frontend por las siguientes razones:

Popularidad	<i>Es una de las bibliotecas frontend más utilizadas a nivel mundial, con una gran comunidad y ecosistema de soporte activo.</i>
Rendimiento	<i>Su arquitectura basada en componentes y Virtual DOM permite renderizados eficientes, ideales para interfaces con actualizaciones frecuentes como dashboards de inventario.</i>
Reutilización	<i>La componentización permite desarrollar elementos de UI reutilizables (tablas, formularios, gráficos), reduciendo el tiempo de desarrollo.</i>

SPA	<i>Funciona como Single Page Application (SPA), lo que mejora la experiencia del usuario al eliminar recargas completas de página.</i>
Integración	<i>Se integra de forma nativa con APIs REST a través de fetch/axios, alineándose perfectamente con el backend Flask propuesto.</i>

3. Capa de Lógica de Negocio – Backend

3.1 Tecnología: Python + Flask

El backend será desarrollado utilizando Python y Flask, encargándose de procesar las reglas de negocio, autenticación, validaciones y manejo de transacciones críticas.

- API REST para comunicación con el frontend.
- Autenticación basada en JWT.
- Validación de datos mediante Marshmallow.
- Gestión modular de productos, ventas y clientes.
- Procesamiento de reportes y alertas de stock mínimo.
- Control de errores y manejo de excepciones.

3.2 Justificación de Python y Flask

La combinación Python + Flask fue seleccionada por los siguientes motivos:

Simplicidad	<i>Flask es un microframework ligero que permite construir APIs REST de manera rápida y clara, sin imponer estructura excesiva al proyecto.</i>
Python	<i>Python es un lenguaje de alta productividad, con sintaxis limpia y amplia disponibilidad de bibliotecas para validación, seguridad y reportes.</i>
Flexibilidad	<i>A diferencia de frameworks más rígidos como Django, Flask permite adoptar únicamente los componentes necesarios, manteniendo el sistema liviano.</i>

SQLAlchemy	<i>La integración con SQLAlchemy como ORM permite abstraer las consultas SQL, reducir errores y proteger contra inyección SQL de forma nativa.</i>
JWT	<i>El soporte nativo para autenticación con JSON Web Tokens permite una gestión de sesiones sin estado (stateless), ideal para APIs RESTful escalables.</i>
Ecosistema	<i>Flask cuenta con extensiones maduras como Flask-JWT-Extended, Flask-Marshmallow y Flask-SQLAlchemy que aceleran el desarrollo.</i>

4. Capa de Datos – Base de Datos

4.1 Tecnología: MySQL 8.0

La persistencia de información será administrada mediante MySQL 8.0, garantizando integridad de datos, relaciones estructuradas y soporte para múltiples usuarios concurrentes. Las entidades principales del sistema son:

- USUARIO – Gestión de acceso y roles del sistema.
- CLIENTE – Información de clientes vinculados a ventas.
- PROVEEDOR – Datos de proveedores para compras.
- PRODUCTO – Catálogo de productos con stock.
- VENTA – Cabecera de transacciones de venta.
- COMPRA – Registro de compras a proveedores.
- DETALLE_VENTA – Líneas de detalle por venta.
- DETALLE_COMPRA – Líneas de detalle por compra.
- CATEGORIA – Clasificación de productos.

4.2 Justificación de MySQL 8.0

MySQL 8.0 fue elegido como motor de base de datos por las siguientes razones:

Madurez	<i>MySQL es uno de los motores de base de datos relacionales más estables y utilizados a nivel mundial, con más de 25 años de desarrollo continuo.</i>
Transacciones ACID	<i>Garantiza atomicidad, consistencia, aislamiento y durabilidad en operaciones críticas como el registro de ventas e inventario.</i>

Rendimiento	<i>MySQL 8.0 incluye mejoras significativas en rendimiento para consultas complejas, joins y operaciones sobre grandes volúmenes de datos.</i>
Costo	<i>Es de código abierto con licencia gratuita para uso comercial en su edición Community, lo cual es ideal para PYMEs con presupuesto limitado.</i>
Compatibilidad	<i>Se integra de forma nativa con SQLAlchemy (el ORM seleccionado para Flask), simplificando la gestión de conexiones y consultas.</i>
Herramientas	<i>Cuenta con herramientas de administración maduras como MySQL Workbench para modelado, monitoreo y administración visual de la base de datos.</i>

5. Relación con los Requerimientos del Sistema

La arquitectura planteada responde directamente a los requerimientos funcionales y no funcionales definidos para el proyecto:

Requerimiento	Implementación Arquitectónica
Gestión de usuarios y roles	JWT y middleware de autorización
Registro de ventas	Módulo de ventas con transacciones ACID
Control de inventario	Actualización automática de stock
Generación de reportes	Frontend React con gráficos y estadísticas
Seguridad	bcrypt, JWT y ORM SQLAlchemy
Escalabilidad	Arquitectura modular desacoplada

6. Seguridad del Sistema

La seguridad es un eje transversal en la arquitectura propuesta. Las medidas implementadas abarcan todas las capas del sistema:

- Autenticación mediante JSON Web Tokens (JWT) con expiración configurable.

- Contraseñas almacenadas con hash bcrypt (algoritmo resistente a ataques de fuerza bruta).
- Protección contra inyección SQL usando SQLAlchemy ORM como capa de abstracción.
- Control de acceso basado en roles (RBAC) para operaciones sensibles.
- Uso de variables de entorno para almacenamiento de credenciales sensibles.
- Validación y sanitización de todos los datos de entrada con Marshmallow.

6.1 Justificación de las Medidas de Seguridad

Las decisiones de seguridad tomadas responden a las siguientes consideraciones:

JWT	<i>Permite autenticación sin estado (stateless), lo que facilita la escalabilidad horizontal del backend y elimina la necesidad de almacenar sesiones en servidor.</i>
bcrypt	<i>Es el estándar de facto para almacenamiento seguro de contraseñas. Incluye salting automático y es computacionalmente costoso de romper por fuerza bruta.</i>
SQLAlchemy	<i>El uso de un ORM elimina la construcción manual de queries SQL, previniendo inyecciones SQL de manera sistemática y sin depender de la diligencia del desarrollador.</i>
Variables de entorno	<i>Separar las credenciales del código fuente es una práctica esencial de seguridad (12-factor app) que evita exponer datos sensibles en repositorios.</i>

7. Flujo Crítico: Registro de Venta

El siguiente flujo describe el proceso completo de registro de una venta, mostrando la interacción entre las tres capas de la arquitectura:

1. El vendedor inicia sesión en el sistema (autenticación JWT).
2. Selecciona productos y cantidades desde la interfaz React.
3. El frontend envía la solicitud mediante la API REST (HTTP POST).
4. El backend Flask valida el token JWT, verifica stock disponible y procesa la transacción.
5. Se registra la venta en MySQL y se actualiza el inventario en la misma transacción ACID.
6. El sistema genera el comprobante correspondiente.

7. Se retorna la respuesta de éxito al usuario con los datos de la venta.

8. Conclusiones

La arquitectura propuesta proporciona una base sólida para el desarrollo del Sistema de Inventario y Ventas orientado a PYMEs. Las decisiones tecnológicas adoptadas — React.js en el frontend, Python + Flask en el backend y MySQL 8.0 como base de datos — no son arbitrarias, sino que responden a criterios técnicos, económicos y estratégicos bien fundamentados:

- React.js garantiza una interfaz moderna, reactiva y de fácil mantenimiento.
- Flask ofrece la flexibilidad necesaria para construir una API REST robusta sin overhead innecesario.
- MySQL asegura la integridad y persistencia de datos con soporte transaccional ACID.
- El conjunto de herramientas seleccionadas es de código abierto, reduciendo costos de licenciamiento para las PYMEs.
- La arquitectura modular facilita la incorporación de nuevas funcionalidades sin comprometer la estabilidad del sistema existente.